Similarly, if you're going to wait for a thread to finish, it might be worth identifying a thread hierarchy, such that a thread waits only for threads lower down the hierarchy. One simple way to do this is to ensure that your threads are joined in the same function that started them, as described in sections 3.1.2 and 3.3.

Once you've designed your code to avoid deadlock, `std::lock()` and `std::lock_guard` cover most of the cases of simple locking, but sometimes more flexibility is required. For those cases, the Standard Library provides the `std::unique_lock` template. Like `std::lock_guard`, this is a class template parameterized on the mutex type, and it also provides the same RAII-style lock management as `std::lock_guard` but with a bit more flexibility.

3.2.6. Flexible locking with `std::unique_lock`

`std::unique_lock` provides a bit more flexibility than `std::lock_guard` by relaxing the invariants; a `std::unique_lock` instance doesn't always own the mutex that it's associated with. First off, just as you can pass `std::adopt_lock` as a second argument to the constructor to have the lock object manage the lock on a mutex, you can also pass `std::defer_lock` as the second argument to indicate that the mutex should remain unlocked on construction. The lock can then be acquired later by calling `lock()` on the `std::unique_lock` object (*not* the mutex) or by passing the `std::unique_lock` object itself to `std::lock()`. Listing 3.6 could just as easily have been written as shown in listing 3.9, using `std::unique_lock` and `std::defer_lock` ❶ rather than `std::lock_guard` and `std::adopt_lock`. The code has the same line count and is essentially equivalent, apart from one small thing: `std::unique_lock` takes more space and is a fraction slower to use than `std::lock_guard`. The flexibility of allowing a `std::unique_lock` instance *not* to own the mutex comes at a price: this information has to be stored, and it has to be updated.

Listing 3.9. Using `std::lock()` and `std::unique_lock` in a swap operation